

Lab 0: Cliente LLM Robusto

Tipo: Guiado | **Duração:** 45 min | **Nível:** Iniciante (warm-up de M1) | **Bloom:** Apply

Objetivo de Aprendizagem

Implementar uma integração programática com APIs de LLM (OpenAI ou Anthropic) usando streaming, retries e saída estruturada validada com Pydantic (objetivo M1-O3).

Este lab roda antes do LAB-001. Ele é o piso técnico do módulo: sem dominar Pydantic, streaming e retry, os alunos da trilha Basic chegam ao tool-use sem as primitivas necessárias. As trilhas Intermediate e Advanced podem fazer um skim em cerca de 20 minutos.

Fluxo do Lab



Fluxo do Lab 0: setup, warm-up, pydantic, stream, retry e error handling

O caminho começa por um setup mínimo de provider, segue para uma chamada de aquecimento que valida a chave, passa por saída estruturada com Pydantic, exercita streaming de tokens e fecha com retry exponencial separando erros que se beneficiam de nova tentativa daqueles que não se beneficiam.

Pré-requisitos

Requisito	Versão
Python	3.10+
openai (SDK)	latest
pydantic	2.x
python-dotenv	1.x

Default da turma: **Gemini Free Tier**. Setup completo de provider, quota e troubleshooting em `notebooks/API-KEYS.pdf`.

Etapa 1. Aquecimento com uma chamada simples

Antes de adicionar complexidade, garanta que o cliente conecta:

```

import os
from openai import OpenAI

client = OpenAI(
    api_key=os.environ['GEMINI_API_KEY'],
    base_url='https://generativelanguage.googleapis.com/v1beta/openai/',
)
MODEL = 'gemini-2.5-flash-lite'

resp = client.chat.completions.create(
    model=MODEL,
    messages=[{'role': 'user', 'content': 'Diga apenas: ping'}],
    temperature=0.0,
)
print(resp.choices[0].message.content)

```

Ponto de verificação: a saída deve imprimir `ping` ou uma variação curta. Erros 401 e 403 indicam chave inválida. Erro 429 indica rate limit, basta esperar 60 segundos.

Etapa 2. Pydantic com saída estruturada

Em vez de parsear texto livre do LLM, force JSON validado por schema. Assim qualquer drift do modelo é detectado cedo, antes de dados inválidos chegarem em produção.

```

from pydantic import BaseModel, Field
from typing import Literal
import json

class Classification(BaseModel):
    sentiment: Literal['positivo', 'negativo', 'neutro']
    confidence: float = Field(ge=0, le=1)
    reasoning: str = Field(max_length=200)

def classify(text: str) -> Classification:
    schema = Classification.model_json_schema()
    prompt = f"""Classifique o sentimento da mensagem abaixo.
Responda APENAS em JSON valido com este schema:
{json.dumps(schema, indent=2)}

Mensagem: {text}
Output: """

    resp = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user', 'content': prompt}],
        response_format={'type': 'json_object'},
        temperature=0.0,
    )
    raw = resp.choices[0].message.content
    return Classification.model_validate_json(raw)

result = classify('Adorei o atendimento, super atencioso!')
print(f'sentiment={result.sentiment}, confidence={result.confidence}')
print(f'reasoning: {result.reasoning}')

```

Pergunta para discussão: o que acontece se o LLM responder com JSON fora do schema, por exemplo um `confidence=1.5` ?

A função `model_validate_json()` levanta `ValidationError`, e isso é desejável. O objetivo é justamente capturar drift do

modelo no momento em que ele acontece. Sem validação, o dado inválido seguiria adiante e contaminaria a aplicação.

Etapa 3. Streaming

Streaming reduz bastante o tempo até o primeiro token. A percepção do usuário fica cerca de três vezes mais rápida mesmo quando o throughput total é o mesmo, porque o texto começa a aparecer imediatamente.

```
def stream_completion(prompt: str) -> str:
    stream = client.chat.completions.create(
        model=MODEL,
        messages=[{'role': 'user', 'content': prompt}],
        stream=True,
    )

    full_text = ''
    for chunk in stream:
        delta = chunk.choices[0].delta.content or ''
        print(delta, end='', flush=True)  # renderiza ao vivo
        full_text += delta
    print()  # newline final
    return full_text

_ = stream_completion('Liste 5 frameworks Python para LLM, 1 linha cada')
```

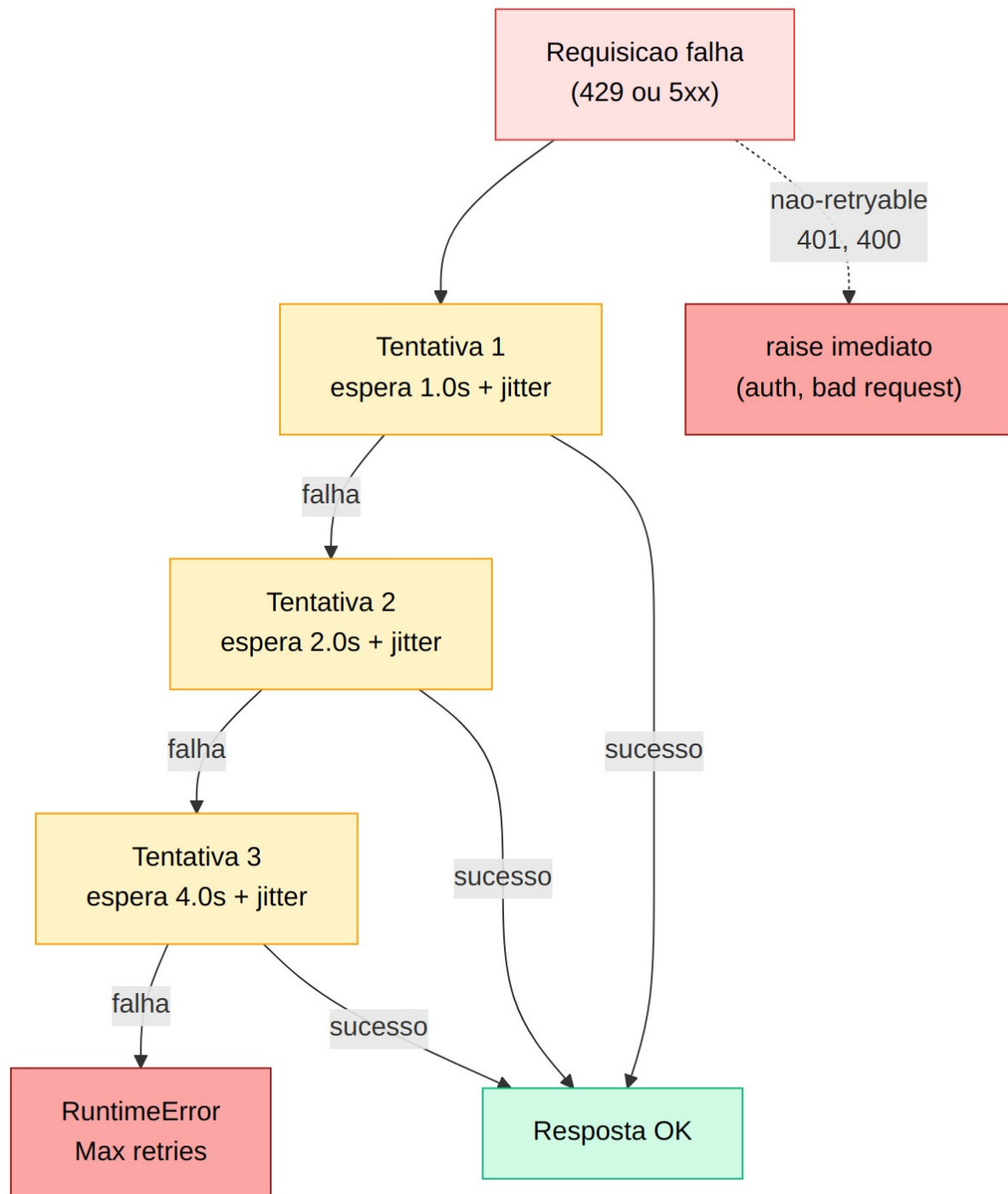
Ponto de verificação: o texto deve aparecer palavra por palavra, nunca como um bloco único.

Pergunta para discussão: por que streaming combina mal com retry?

O problema é que, se a conexão cair no meio do stream, parte do conteúdo já foi exibida ao usuário. Um retry gera conteúdo diferente, porque o LLM não é determinístico, e isso causa flicker ou continuação inconsistente. Em uma UI de chat, a estratégia razoável é aceitar a quebra e exibir uma mensagem de reconexão. Em batch processing, a saída melhor é não usar streaming e adotar retry idempotente. A combinação de streaming com retry idempotente só funciona quando o servidor expõe um cursor, o que é raro.

Etapa 4. Retry com exponential backoff e jitter

Em produção, chamadas falham por rate limit, problemas de rede ou erros 5xx do servidor. Sem retry, o produto vira frágil. Retry sem backoff produz thundering herd e piora a situação.



Fluxo de retry: backoff exponencial em 1s, 2s, 4s com jitter, separando erros retryable de não-retryable

A figura acima mostra os três elementos centrais: separação entre erros que cabem retry (rate limit, conexão, timeout, 5xx) e erros que não cabem (autenticação, bad request), espera que cresce de forma exponencial (1s, 2s, 4s) e um jitter aleatório somado ao delay para evitar que vários clientes batam no servidor exatamente no mesmo instante.

```

import random
import time
from openai import (
    APIConnectionError,
    APITimeoutError,
    RateLimitError,
    InternalServerError,
    AuthenticationError,
    BadRequestError,
)

RETRYABLE = (RateLimitError, APIConnectionError, APITimeoutError, InternalServerError)
NON_RETRYABLE = (AuthenticationError, BadRequestError)

def call_with_retry(prompt: str, max_attempts: int = 3) -> str:
    base_delay = 1.0
    last_error: Exception | None = None

    for attempt in range(1, max_attempts + 1):
        try:
            resp = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': prompt}],
                temperature=0.0,
            )
            return resp.choices[0].message.content or ''

        except RETRYABLE as e:
            last_error = e
            if attempt == max_attempts:
                break

            delay = base_delay * (2 ** (attempt - 1))
            jitter = random.uniform(0, delay * 0.5)
            wait = delay + jitter
            print(f' [retry {attempt}] {type(e).__name__}: aguardando {wait:.1f}s')
            time.sleep(wait)

        except NON_RETRYABLE as e:
            # Auth invalida ou input malformatado: re-tentar nao adianta
            raise

    raise RuntimeError(f'Max retries ({max_attempts}) excedidas') from last_error

# Teste manual (so cai em erro real se voce derrubar a internet):
result = call_with_retry('Diga: ok')
print(f'Resultado: {result}')

```

Etapa 5. Combinar Pydantic com retry em um pipeline único

A versão robusta junta validação de schema e tolerância a falhas transitórias em uma única função. É a forma que normalmente vai para produção.

```

def robust_classify(text: str, max_attempts: int = 3) -> Classification:
    schema = Classification.model_json_schema()
    prompt = f"""Classifique o sentimento. Responda apenas em JSON:
Schema: {json.dumps(schema)}
Mensagem: {text}
Output: """

    base_delay = 1.0
    for attempt in range(1, max_attempts + 1):
        try:
            resp = client.chat.completions.create(
                model=MODEL,
                messages=[{'role': 'user', 'content': prompt}],
                response_format={'type': 'json_object'},
                temperature=0.0,
            )
            raw = resp.choices[0].message.content or ''
            return Classification.model_validate_json(raw)

        except RETRYABLE as e:
            if attempt == max_attempts:
                raise

            wait = base_delay * (2 ** (attempt - 1)) + random.uniform(0, 0.5)
            print(f' [retry {attempt}] {type(e).__name__}: {wait:.1f}s')
            time.sleep(wait)

        except NON_RETRYABLE:
            raise

    raise RuntimeError('unreachable')

# Bateria de 3 mensagens
for msg in [
    'Adorei o atendimento, super atencioso!',
    'Demoraram 3 dias pra responder e nada.',
    'Recebi o produto.',
]:
    out = robust_classify(msg)
    print(f'{msg[:40]:42s} -> {out.sentiment} ({out.confidence:.2f})')

```

Verificação

A chamada de aquecimento funciona e confirma que a chave é válida.

`Classification.model_validate_json()` aceita JSON válido e rejeita JSON fora do schema.

O streaming exibe o texto palavra por palavra e não em bloco.

O retry separa corretamente erros que cabem nova tentativa (rate limit, rede) dos que não cabem (autenticação, bad request).

O pipeline combinado classifica as três mensagens com sucesso.

Troubleshooting

Problema 1. `ValidationError` em quase toda chamada

A causa quase sempre é o LLM não respeitar o schema. Reforce no prompt que a resposta deve ser apenas JSON, sem prefixos ou comentários. Quando o provider suportar, use `response_format={'type': 'json_object'}`. Mantenha `temperature=0.0` e, se ainda assim houver desvio, adicione um ou dois exemplos few-shot mostrando exatamente o formato esperado.

Problema 2. Streaming trava no meio

A causa típica é idle timeout, situação comum em redes corporativas. A correção mais simples é adicionar `timeout=60` ao cliente. Outra opção é detectar via `time.time()` que mais de 30 segundos se passaram sem chegar nenhum chunk e nesse caso reconectar (sem usar retry idempotente, apenas registrando o ocorrido em log e seguindo adiante).

Problema 3. Loop de retry infinito mesmo com `max_attempts`

Isso costuma significar que o erro real não está em `RETRYABLE` mas está sendo capturado por algum outro `except` que continua o loop. O `raise` final no bloco `NON_RETRYABLE` é essencial. Sem ele o erro é silenciado e o loop continua tentando.

Próximo passo

Com o notebook 00 fechado, abra o notebook 01 (LAB-001 Agent CLI). As primitivas exercitadas aqui (Pydantic, retry e structured output) reaparecem lá compondo o agente com tool-use.

Referências

- [OpenAI Python SDK, Errors and Retries](#)
- [Pydantic v2, Models](#)
- [Exponential Backoff and Jitter, AWS Architecture Blog](#)